Reviewer Pack — Polymer Cluster-Expansion Convergence Radius Bounds Tree-Res…

Entry 3905f361f983 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 21:20:04 UTC

Polymer Cluster-Expansion Convergence Radius Bounds Tree-Res Size

Entry ID: `3905f361f983` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-17 21:20:04 UTC

1. Conjecture

Field A (mathematical branch): Statistical mechanics — Fernandez–Procacci / Kotecký–Preiss cluster-expansion convergence radius for abstract polymer models (Mayer 1937; Kotecký–Preiss 1986; Fernandez–Procacci 2007; Bissacot–Fernandez–Procacci 2010). An analytic structural invariant of finite graphs essentially absent from proof complexity outside loose Lovász-Local-Lemma analogies; an arXiv search ‘cluster expansion’ AND (‘Resolution refutation’ OR ‘DPLL’ OR ‘Frege’ OR ‘proof complexity’) returns no direct hits and <5 adjacent papers, none using FP fixed-point activities as a CNF invariant. **Field B** (complexity object): Tree-like Resolution / DPLL refutation tree size t*(F) for unsatisfiable 3-CNFs, in the Ben-Sasson–Wigderson–Krajíček bounded-arithmetic regime where ¬F is Σ^b_0 and log_2 t*(F) controls V^0-witnessing complexity — a Frege/Extended-Frege stepping stone via the Pudlák–Buss feasible-interpolation and Krajíček EF-depth simulation chain.

Statement:

> For an unsatisfiable 3-CNF F on n variables with clauses C_1,…,C_m, build the clause-overlap graph G(F) (vertices = clauses; edge iff the two clauses share a variable) and let μ*(F) ∈ (0,1]^m be the unique fixed point of the Fernandez–Procacci iteration μ_v ← 1/(1 + Σ_{u ~ v} μ_u) initialised at μ_v^(0)=1; define the polymer convergence defect Λ(F) := −Σ_v log_2 μ_*v. We conjecture there is an absolute constant c ≥ 1/8 such that log_2 t*(F) ≥ c · Λ(F) for every unsatisfiable 3-CNF F, with the bound near-tight on random 3-CNFs at α ≈ 4.26. A single unsatisfiable F with log_2 t*(F) < (1/8)·Λ(F) falsifies the conjecture.

Rationale (proposer’s reasoning):

> Cluster-expansion convergence is the analytic dual of clause-overlap expansion: the FP activities μ_v contract precisely when local clause interactions admit a convergent Mayer series, mirroring the same expansion mechanism underlying Ben-Sasson–Wigderson width lower bounds — but packaged as a single-number invariant of the SYNTACTIC formula, not of any truth-table, so it dodges the Razborov–Rudich naturalization route. The construction is non-arithmetizing (the fixed point is a graph-combinatorial invariant of the CNF symbol level, not a polynomial extension of the Boolean ring), avoiding the algebrization wall hit by GCT cubic-form attempts. If correct, Λ(F) would supply a physics-style ‘free-energy’ depth measure for DPLL and, via interpolation, the first quantitative structural bridge from cavity-method analysis to Frege/EF lower bounds.

Taxonomy category: `PROOF_COMPLEXITY_EXT_FREGE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0a4d5dd8a3e16d87`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across all 210 unsatisfiable 3-CNFs (30 seeds × 7 sizes n∈{10,14,18,22,26,30,34}, α=4.26), compute r(F)=log₂t*(F)/Λ(F). SUPPORTED iff min_seed r(F) ≥ 0.125 AND mean r(F) ≤ 0.5; FALSIFIED iff any single instance yields r(F) < 0.125.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.88	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `cluster expansion Fernandez Procacci resolution proof complexity` - `Kotecky Preiss polymer model tree-like resolution lower bound 3-CNF` - `clause variable graph fixed point activity DPLL refutation size random 3-SAT`

Top relevant hits considered: - [http://arxiv.org/abs/1705.04385v2] A correction to a remark in a paper by Procacci and Yuhjtman: new lower bounds for the convergence radius of the virial - [http://arxiv.org/abs/math-ph/0605041v2] Cluster expansion for abstract polymer models. New bounds from an old approach - [http://arxiv.org/abs/1002.3261v1] On the convergence of cluster expansions for polymer gases - [http://arxiv.org/abs/1904.03250v1] Velocity jump in the crack propagation induced on a semi-crystalline polymer sheet by constant-speed stretching - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer - [http://arxiv.org/abs/1710.07476v6] Planar 3-SAT with a Clause/Variable Cycle - [http://arxiv.org/abs/1908.05361v2] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.3s

5.1 Generated Python source

```python
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_3cnf(n, alpha):
```

```
    m = int(alpha * n)
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, 3)
        if random.choice([True, False]):
            clause[0] = -clause[0]
        if random.choice([True, False]):
            clause[1] = -clause[1]
        if random.choice([True, False]):
            clause[2] = -clause[2]
        clauses.append(clause)
    return clauses

def is_unsatisfiable(clauses):
    n = max(max(abs(lit) for lit in clause) for clause in clauses)
    assignments = [None] * (n + 1)

    def dpll(clauses, assignments):
        if not clauses:
            return True
        if any(not clause for clause in clauses):
            return False

        for clause in clauses:
            if len(clause) == 1:
                lit = clause[0]
                var = abs(lit)
                if assignments[var] is not None and assignments[var] != (lit > 0):
                    return False
                assignments[var] = lit > 0
                new_clauses = [c for c in clauses if lit not in c]
                new_clauses = [[l for l in c if l != -lit] for c in new_clauses]
                if dpll(new_clauses, assignments.copy()):
                    return True
                assignments[var] = None
                return False

        var = max(range(1, n + 1), key=lambda v: sum(1 for clause in clauses if v in clause or -v
        for val in [True, False]:
            assignments[var] = val
            new_clauses = [c for c in clauses if var not in c and -var not in c]
            new_clauses = [[l for l in c if l != (var if val else -var)] for c in new_clauses]
            if dpll(new_clauses, assignments.copy()):
                return True
            assignments[var] = None
        return False

    return not dpll(clauses, assignments)

def build_clause_overlap_graph(clauses):
    graph = defaultdict(set)
    for i, clause1 in enumerate(clauses):
        for j, clause2 in enumerate(clauses):
            if i != j and any(lit in clause2 or -lit in clause2 for lit in clause1):
                graph[i].add(j)
```

```
    return graph

def compute_fixed_point(graph, m):
    mu = [1.0] * m
    for _ in range(500):
        new_mu = [0.0] * m
        for v in range(m):
            sum_neighbors = sum(mu[u] for u in graph[v])
            new_mu[v] = 1.0 / (1.0 + sum_neighbors)
        if max(abs(new_mu[v] - mu[v]) for v in range(m)) < 1e-9:
            break
        mu = new_mu
    return mu

def compute_lambda(mu):
    return -sum(math.log2(mu_v) for mu_v in mu)

def count_dpll_leaves(clauses):
    n = max(max(abs(lit) for lit in clause) for clause in clauses)
    assignments = [None] * (n + 1)

    def dpll_count(clauses, assignments):
        if not clauses:
            return 1
        if any(not clause for clause in clauses):
            return 0

        for clause in clauses:
            if len(clause) == 1:
                lit = clause[0]
                var = abs(lit)
                if assignments[var] is not None and assignments[var] != (lit > 0):
                    return 0
                assignments[var] = lit > 0
                new_clauses = [c for c in clauses if lit not in c]
                new_clauses = [[l for l in c if l != -lit] for c in new_clauses]
                return dpll_count(new_clauses, assignments.copy())

        var = max(range(1, n + 1), key=lambda v: sum(1 for clause in clauses if v in clause or -v
        count = 0
        for val in [True, False]:
            assignments[var] = val
            new_clauses = [c for c in clauses if var not in c and -var not in c]
            new_clauses = [[l for l in c if l != (var if val else -var)] for c in new_clauses]
            count += dpll_count(new_clauses, assignments.copy())
            assignments[var] = None
        return count

    return dpll_count(clauses, assignments)

def run_trial(seed):
    random.seed(seed)
    n_sizes = [10, 14, 18, 22, 26, 30, 34]
    alpha = 4.26
    results = []
```

```
    for n in n_sizes:
        clauses = generate_3cnf(n, alpha)
        if not is_unsatisfiable(clauses):
            continue

        graph = build_clause_overlap_graph(clauses)
        m = len(clauses)
        mu = compute_fixed_point(graph, m)
        Lambda = compute_lambda(mu)
        t_star = count_dpll_leaves(clauses)

        if Lambda <= 0:
            continue

        r = math.log2(t_star) / Lambda
        results.append(r)

    if not results:
        return {
            "metric_name": "r(F)",
            "metric_value": 0.0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid instances generated"
        }

    min_r = min(results)
    mean_r = sum(results) / len(results)
    conjecture_holds = min_r >= 0.125 and mean_r <= 0.5
    counterexample = "" if conjecture_holds else f"min_r={min_r} < 0.125"

    return {
        "metric_name": "r(F)",
        "metric_value": mean_r,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    metric_values = []
    conjecture_holds_counts = 0
    counterexamples = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        metric_values.append(result["metric_value"])
        if result["conjecture_holds"]:
            conjecture_holds_counts += 1
        if result["counterexample"]:
            counterexamples.append(result["counterexample"])

    mean_metric = sum(metric_values) / len(metric_values) if metric_values else 0.0
    std_metric = (sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)) ** 0.5
```

```
    support_fraction = conjecture_holds_counts / len(seeds) if seeds else 0.0

    if counterexamples:
        print(f"RESULT: FALSIFIED counterexample={counterexamples[0]} first_failing_seed={seeds[co
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
es_tested': 0, 'conjecture_holds': False, 'counterexample': 'No valid instances generated'}
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
TRIAL: {'metric_name': 'r(F)', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False
RESULT: FALSIFIED counterexample=No valid instances generated first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The stdout shows zero valid instances were generated across all trials (instances_tested: 0, metric_value: 0.0) and the run terminated as FALSIFIED with counterexample 'No valid instances generated'. This is not evidence about the conjecture at all — it's a construction gap / generator bug, so neither SUPPORTED nor FALSIFIED can be concluded. The empirical test never actually evaluated log_2 t*(F) vs c·Λ(F) on a single unsatisfiable 3-CNF.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test generated zero valid instances across all 210 trials (instances_tested: 0, metric_value: 0.0), so the conjecture was never empirically evaluated. The 'FALSIFIED' RESULT line reflects a generator/construction failure, not a genuine counterexample to the bound. | next: Fix the unsatisfiable 3-CNF generator (e.g., sample random 3-CNFs at α=4.26 and filter via a SAT solver, or use known unsat constructions like PHP_n+1^n encodings) so that t*(F) and Λ(F) can actually be computed for all 210

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	279535
2	preregistration	claude_max	opus	0	0	5478
3	novelty	claude_max	opus	0	0	4266
4	novelty	claude_max	opus	0	0	6614
5	test_gen	mistral	codestral-latest	0	0	314499
6	test_gen	mistral	codestral-latest	0	0	310476
7	test_gen	mistral	codestral-latest	0	0	314629
8	test_gen	mistral	codestral-latest	0	0	313359
9	critic	claude_max	opus	0	0	8052
10	judge	claude_max	opus	0	0	7487

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1564396 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/3905f361f983.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3905f361f983.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3905f361f983.tar.gz` (if generated)